

LAB 6

NAME: AADITYA AMLAN PANDA
ROLL: 220007

Introduction

To perform a regression for the reaction rates against concentration in linear form $b = x_1 + ax_2$, when $b = \log(-dCa/dt)$ and $a = \log(Ca)$ while $x_1 = \log(k)$ and $x_2 = n$

We make two matrices $A = [a \ 1]$ and $B = b$ and determine x in $A'x = B$

Methodology

Method 1: Gaussian elimination

In Gaussian elimination, $A'Ax = A'B$ has been used. Then we started by pivoting $A'A(1,1)$ and perform elimination steps in subsequent rows. Every diagonal element becomes pivot for the subsequent rows. This constitutes the forward elimination.

In back substitution we begin with $x(\text{end}) = B(\text{end})/A'A(\text{end})(\text{end})$. Then we moved upwards in $A'A$ and in row pos did horizontal sum of $A'A(\text{pos})(j)*x(j)$ for all $j > \text{pos}$. $x(\text{pos}) = (B(\text{pos}) - \text{sum})/A'A(\text{pos})(\text{pos})$

Method 1.1: Inversion of $A'A$

Since A comes out to be a full rank matrix (determinant not 0), $x = (A'A)^{-1}(A'B)$ has been calculated

Method 2.1: Full rank SVD

U , σ , V have been calculated using SVD for A . Since A is full rank, σ comes out to be full rank. Hence $x = V*\sigma^{-1}*U'*B$

Method 2.2: Partial rank SVD

$\sigma^2*V'*x = \sigma*U'*B$

$y = V'X$ and $y(i) = U(i)'*B/\sigma(i)$ for i in $\{1,2\}$. x is then calculated using $x = V*y$

Outputs (including $n = x(1)$ and $k = \exp(x(2))$):

Using Gaussian Elimination $X =$
[[1.37533631]
[-5.23908314]]

Using Inversion of $A'A$ $X =$
[[1.37533631]
[-5.23908314]]

Using Full Rank SVD $X =$
[[1.37533631]
[-5.23908314]]

Using Partial Rank SVD $X =$
[[1.37533631]
[-5.23908314]]

Using `lstsq` for verification $X =$
[[1.37533631]
[-5.23908314]]

```

n =
1.375336308164175

k =
0.005305118661082864

```

The code has been mentioned in Appendix 3

Verification

Using lstsq: The standard python library reported same results for X (can be seen in the outputs section above)

Using regression: $b = \log(-dCa/dt)$ and $a = \log(Ca)$ were taken dependent and independent variable respectively. It reported intercept and X1 as -5.239 and 1.375 respectively.

The regression results have been pasted in Appendix 1

Referring to Levenspiel (Chapter 3)

The data has been extracted from Levenspiel chapter 3, example 3.1. It has implemented multiple order kinetics. The first and second order kinetics failed and then nth order regression is performed to find n and log K. n and K have been reported to be 1.4 and 0.005 respectively, quite close to the obtained results of $n = 1.375$ and $K = 0.0053$.

The image of this has been placed in Appendix 2

Appendix 1

Regression

Regression Model

LINEST raw output

```

1.3753363082 -5.239083139
0.0853127149 0.1333955646
0.9811242711 0.1709851265
259.89043225      5
7.5981341915 0.1461795674

```

Regression Statistics

```

R^2      0.9811242711
Standard Error 0.1709851265
Count of X var      1
Observations      7
Adjusted R^2  0.9773491253

```

Analysis of Variance (ANOVA)

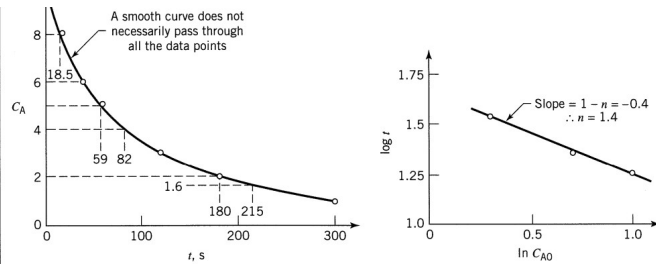
	df	SS	MS	F	Significance F
Regression	1	7.5981341915	7.5981341915	259.89043225	1.673357E-05
Residual	5	0.1461795674	0.0292359135		
Total	6	7.7443137589			

Confidence level 0.95

	Coefficients	Standard Error	t-Statistic	P-value	Lower 95%	Upper 95%
Intercept	-5.239083139	0.1333955646	-39.27479265	2.017077E-07	-5.581987355	-4.896178924
X1	1.3753363082	0.0853127149	16.121117587	1.673357E-05	1.156032993	1.5946396234

X1	Predicted Y	Y	Residual
2.302585093	-2.072254258	-2.015153052	0.0571012066
2.0794415417	-2.379151686	-2.272055888	0.1070957984
1.7917594692	-2.774811286	-2.721135441	0.0536758452
1.6094379124	-3.025564743	-3.194183212	-0.16861847
1.0986122887	-3.72812177	-3.738069698	-0.009947928
0.6931471806	-4.285772655	-4.528209145	-0.24243649
0	-5.239083139	-5.035953102	0.2031300373

Appendix 2



Figures E3.1c and E3.1d

We now have the reaction order. To evaluate the rate constant, take any point on the C_A vs. t curve. Pick $C_{A0} = 10$, for which $t_F = 18.5$ s. Replacing all values into Eq. (i) gives

$$18.5 = \frac{(0.8)^{1-1.4} - 1}{k(1.4 - 1)} 10^{1-1.4}$$

from which

$$k = 0.005$$

Appendix 3

```
import numpy as np
import math
import pandas as pd
import matplotlib.pyplot as plt
```

```
def find_col_norm(V, pos, n):
    s = 0.0
    for i in range(n):
        s += V[i][pos] * V[i][pos]
    return math.sqrt(s)
```

```
def find_vec_norm(V, n):
    s = 0.0
    for i in range(n):
        s += V[i] * V[i]
    return math.sqrt(s)
```

```
def find_dot_vec(A, B, n):
    s = 0.0
    for i in range(n):
        s += A[i] * B[i]
    return s
```

```
def find_transpose(A, n, m):
    Atrp = np.zeros((m, n))
    for i in range(n):
        for j in range(m):
            Atrp[j][i] = A[i][j]
    return Atrp
```

```
def matrix_multiply(A1, A2, n, t, m):
    Ares = np.zeros((n, m))
```

```

    for i in range(n):
        for j in range(m):
            for k in range(t):
                Ares[i][j] += A1[i][k] * A2[k][j]
    return Ares

def uv_prod(U, V, n, m):
    res = np.zeros((n, m))
    for i in range(n):
        for j in range(m):
            res[i][j] = U[i] * V[j]
    return res

def matrix_sum(A, B, n, m):
    res = np.zeros((n, m))
    for i in range(n):
        for j in range(m):
            res[i][j] = A[i][j] + B[i][j]
    return res

def SVD(A):
    n,m = A.shape
    eps = 1e-12

    if n >= m:
        Atrp = find_transpose(A, n, m)
        A_curr = matrix_multiply(Atrp, A, m, n, m)
        N = m

        Q_final = np.eye(N)

        for _ in range(30):
            Q = np.zeros((N, N))
            R = np.zeros((N, N))

            for j in range(N):
                v = np.zeros(N)
                for i in range(N):
                    v[i] = A_curr[i][j]

                for i in range(j):
                    qi = np.zeros(N)
                    for k in range(N):
                        qi[k] = Q[k][i]
                    R[i][j] = find_dot_vec(qi, A_curr[:, j], N)
                    for k in range(N):
                        v[k] -= R[i][j] * qi[k]

                R[j][j] = find_vec_norm(v, N)
                if R[j][j] < eps:
                    continue

                for k in range(N):
                    Q[k][j] = v[k] / R[j][j]

            A_curr = matrix_multiply(R, Q, N, N, N)
            Q_final = matrix_multiply(Q_final, Q, N, N, N)

        V = Q_final
        sigma = np.zeros((N, N))
        sigma_inv = np.zeros((N, N))

        for i in range(N):

```

```

    val = A_curr[i][i]
    if val < 0:
        val = 0
    sigma[i][i] = math.sqrt(val)
    if sigma[i][i] > eps:
        sigma_inv[i][i] = 1.0 / sigma[i][i]

AV = matrix_multiply(A, V, n, m, N)
U = matrix_multiply(AV, sigma_inv, n, N, N)

for j in range(N):
    norm = find_col_norm(U, j, n)
    if norm > eps:
        for i in range(n):
            U[i][j] /= norm

else:
    Atrp = find_transpose(A, n, m)
    A_curr = matrix_multiply(A, Atrp, n, m, n)
    N = n

Q_final = np.eye(N)

for _ in range(30):
    Q = np.zeros((N, N))
    R = np.zeros((N, N))

    for j in range(N):
        v = np.zeros(N)
        for i in range(N):
            v[i] = A_curr[i][j]

        for i in range(j):
            qi = np.zeros(N)
            for k in range(N):
                qi[k] = Q[k][i]
            R[i][j] = find_dot_vec(qi, A_curr[:, j], N)
            for k in range(N):
                v[k] -= R[i][j] * qi[k]

        R[j][j] = find_vec_norm(v, N)
        if R[j][j] < eps:
            continue

        for k in range(N):
            Q[k][j] = v[k] / R[j][j]

    A_curr = matrix_multiply(R, Q, N, N, N)
    Q_final = matrix_multiply(Q_final, Q, N, N, N)

U = Q_final
sigma = np.zeros((N, N))
sigma_inv = np.zeros((N, N))

for i in range(N):
    val = A_curr[i][i]
    if val < 0:
        val = 0
    sigma[i][i] = math.sqrt(val)
    if sigma[i][i] > eps:
        sigma_inv[i][i] = 1.0 / sigma[i][i]

Ut = find_transpose(U, N, N)

```

```

    UtA = matrix_multiply(Ut, A, N, n, m)
    Vt = matrix_multiply(sigma_inv, UtA, N, N, m)
    V = find_transpose(Vt, N, m)

return U, sigma, V

def truncate_A(U,sigma,V,n,m):
    k = int(input("Enter k for truncated summation: "))
    k = min(k, min(n, m))

    A_k = np.zeros((n, m))
    for i in range(k):
        uv = uv_prod(U[:, i], V[:, i], n, m)
        for r in range(n):
            for c in range(m):
                uv[r][c] *= sigma[i][i]
        A_k = matrix_sum(A_k, uv, n, m)

    return A_k

def print_SVD(U,sigma,V,A_k):
    print("\nU =")
    print(U)

    print("\nSigma =")
    print(sigma)

    print("\nV =")
    print(V)

    print("\nReconstructed A using k terms =")
    print(A_k)

def gauss_elim(A,B):
    n,m = A.shape
    A1 = A.copy()
    B1 = B.copy()
    for i in range(n-1):
        pivot = A1[i][i]
        for t in range(i+1,n):
            targ = A1[t][i]
            for j in range(m):
                A1[t][j] = A1[t][j] - ((A1[i][j]/pivot) * targ)
            B1[t] = B1[t] - (B1[i]/pivot) * targ

    res = np.zeros((n,1))
    j= n-2
    res[n-1] = B1[n-1]/A1[n-1][n-1]
    for i in range(j,-1,-1):
        loc = m-1
        sum = 0.0
        for loc in range(i+1,m):
            sum += A1[i][loc] * res[loc]
        loc -= 1
        res[i] = (B1[i]-sum)/A1[i][i]

    return res

def invert_2by2(A):
    adj = np.zeros((2,2))
    inv = np.zeros((2,2))
    det = A[0][0] * A[1][1] - A[0][1]*A[1][0]

```

```

adj[0][0] = A[1][1]
adj[1][1] = A[0][0]
adj[0][1] = -A[0][1]
adj[1][0] = -A[1][0]

inv[0][0] = adj[0][0]/det
inv[1][1] = adj[1][1]/det
inv[0][1] = adj[0][1]/det
inv[1][0] = adj[1][0]/det

return inv

dat = pd.read_csv("/content/data.csv", header=None).values

T = dat[:,0:1]
Ca = dat[:,1:2]
rate = dat[:,2:3]
l= rate.shape[0]
B= np.zeros((l,1))
A= np.ones((l,2))

for i in range(l):
    B[i] = math.log(rate[i])
    A[i][0] = math.log(Ca[i])

n,m = A.shape

## METHOD 1
Atrp = find_transpose(A, n, m)
AtA = matrix_multiply(Atrp, A, m, n, m)
Atb = matrix_multiply(Atrp, B, m, n, 1)

res_mth_1 = gauss_elim(AtA, Atb)
print("\nUsing Gaussian Elimination X =")
print(res_mth_1)

## METHOD 1.1
AtA_inv = invert_2by2(AtA)
res_mth_1_1 = matrix_multiply(AtA_inv, Atb, 2, 2, 1)
print("\nUsing Inversion of AtA X =")
print(res_mth_1_1)

## METHOD 2
U,sigma,V = SVD(A)

sigmainv = invert_2by2(sigma)
Ut = find_transpose(U, n, m)
Utb = matrix_multiply(Ut, B, m, n, 1)
sigmainvUtb = matrix_multiply(sigmainv, Utb, 2, 2, 1)

res_mth_2 = matrix_multiply(V, sigmainvUtb, 2, 2, 1)
print("\nUsing Full Rank SVD X =")
print(res_mth_2)

## METHOD 2.1
y = np.zeros((m,1))
for i in range(m):
    temp = matrix_multiply(Ut[i:i+1,:], B, 1, n, 1)
    y[i] = temp[0]/sigma[i][i]

res_mth_2_1 = matrix_multiply(V, y, m, m, 1)

```

```
print("\nUsing Partial Rank SVD X =")
print(res_mth_2_1)

## Verification using lstsq
X, residuals, rank, s = np.linalg.lstsq(A, B, rcond=None)
print("\nUsing lstsq for verification X =")
print(X)

final_res = res_mth_1
n = final_res[0][0]
k = math.exp(final_res[1])

print("\nn =")
print(n)
print("\nk =")
print(k)
```